

MCP as a Platform

What I Learned Building a Portfolio of MCP Servers

One server teaches you the protocol. A portfolio teaches you what MCP actually is — six open-source servers, two editor extensions, and the mistakes I only found by building the next one.

Every example here is in one small server you can clone tonight: **MCP Demo** — github.com/markusvankempen/mcp-ticket-demo

Markus van Kempen · Executive Architect & Venture Capitalist in Residence, IBM

mvankempen@ca.ibm.com

markus.van.kempen@gmail.com

markusvankempen.github.io

Different systems. The same six questions.

Cloud containers

Support ticketing

Asset management

Quantum circuits

Agent orchestration

What do I call this?

Names that felt cosmetic at a handful of tools became the interface at dozens.

Is it done when it returns 200?

The call succeeded. The outcome was still wrong.

Where does this actually run?

My laptop and someone else's container are not the same place.

Who is allowed to call it?

The protocol describes the tool. It never says who may use it.

Does it work out of the box?

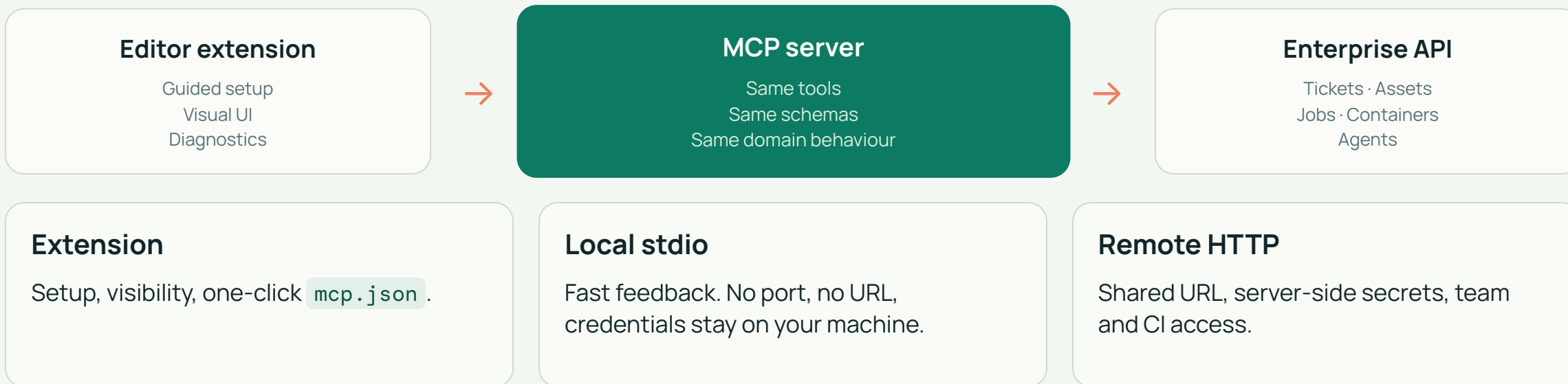
For two readers, actually — the developer installing it and the model calling it.

How does anyone find it?

I published on every channel there is. People only find a server if they already know its name.

One capability. Three ways to run it.

For this talk, imagine we are exposing a Zendesk-style ticketing system to an AI agent – search, create, comment. That is **MCP Demo**.



**The extension is the control plane.
The MCP server is the capability.**

The first version is deceptively simple.

```
client → MCP server → ticketing API
      |
      +-- tool: search_tickets
```

That is a working MCP server. Then the real requests arrive:

```
create one, comment on one, close one
a safer default than "return everything"
what "no results" means, so it stops retrying
who is allowed to call the write tools
and how anyone finds the server at all
```

At a handful of tools, naming is the interface at dozens.

The early question

"Can the model call this API?"

The later question

"Can a developer understand and operate this tool safely?"

```
search_tickets(query, status, requester_email, limit)
```

is easier to use — and safer — than:

```
request(path, method, query, body)
```

The second one is less code for me
and every decision for the model.

More tools didn't mean more capability.

– BEFORE –

```
query_records  
query_with_filter  
query_with_fields  
query_object_structure  
query_work_orders  
query_assets  
query_locations  
...
```

Every name correct.
The model still picked wrong.

– AFTER –

```
list_schemas    what can I query?  
get_schema      what fields exist?  
run_query       ask the question
```

Three tools instead of seven.
Selection stopped being a coin flip.

Technically precise.
Operationally confusing.

The hardest tool wasn't the complicated one.

SHIPPED

`create_ticket`

Two parameters. Returned `201 Created` every single time.

BROKE

Wrong owner

The requester was my API service account, so the platform mailed every reply to the bot — not the customer.

FIXED

Requester attribution

Create on behalf of the real customer, so the whole downstream email thread routes correctly.

Try it: call `create_ticket` in `mcp-ticket-demo` without `requester_email`, then look at who owns the ticket.

**A tool isn't done when the API call succeeds.
It's done when the next thing that happens is right.**

A tool is a UI for an agent.

An extension is a UI for the developer.

Import

Read the `mcp.json` that's already in the workspace instead of asking anyone to retype it.

Diagnose

Credential, health, `tools/list`, then one real call — each step pass or fail, in order.

Test

Call any tool with real arguments and read the raw response, without leaving the editor.

It turns "something is broken" into "this step failed, here's why, here's what to try next."

Zero tools discovered. No error. No warning.

What I registered

```
command: node  
args: ["/Users/markus/mcp-ticket-demo/..."]
```

→ 0 tools discovered

The runner is a cloud container.
It looked for /Users/markus/... → not there.
A missing file reads as an empty tool list.

What actually works

```
command: node  
args: ["src/index.js"]  
cwd: relative to the package I shipped
```

→ 9 tools discovered ✓

Same code. Same command.
Different **execution context**.

Secrets move

Off the laptop and into the platform.

Diagnostics stop being optional

`/health`, `/test`, `/admin` earn their keep.

The URL isn't stable

A new project means a new hostname.
Never paste it into `mcp.json`.

The failures that taught me the most.

What I saw	What was actually wrong	What I changed
0 tools discovered	I gave a cloud runner a path from my laptop	Ship the package, run from a relative path
A strange vendor error, never a 404	Wrong API host — the bad hostname answered with something else entirely	Pin the confirmed host and assert it at startup
Auth looked fine, every call failed	A required tenant header was missing	Check required headers before the first call, not after
mcp.json worked yesterday	Redeploying gave the service a new hostname	Read the endpoint after deploy; never paste a URL by hand
The model picked the wrong tool	Seven near-identical <code>query_*</code> names	One query tool plus schema discovery
Client hung, or a JSON parse error	<code>console.log("connected")</code> wrote into the stdio stream	Log diagnostics to <code>stderr</code> — <code>console.error</code>

Most of these failed **silently**. The parse error named nothing useful either. That's the pattern I'd warn my past self about.

Who is allowed to call it?

The protocol doesn't say.

```
# What I shipped first
```

```
403 Forbidden
```

```
The model retried.  
Then retried again.  
Then told the user the  
system was "unavailable."
```

```
# What it says now
```

```
create_ticket needs "write".  
This key has [read].
```

```
Issue one with that scope  
on /admin → API keys.
```

```
→ it asks for a key  
instead of looping.
```

A locked tool still shows up in `tools/list`.
The refusal is the only place left to explain yourself.

Three modes in the demo: `off` on your laptop, `write` so reads stay open and writes need a key, `all` for anything with a public URL. Discovery stays open in every mode — a client that can't ask "what do you need from me?" can only guess.

Copy the boring parts. Leave the tools alone.

Server two made me want one library for everything. The only things that were actually the same: the folder, the README, how it starts, how it fails. Searching a ticket is not deploying a container.

Copy this

- Same folder and README shape
- Same env and startup checks
- Same error shape and logs
- Same version notes
- One test that it starts

Don't copy this

- Ticket search is not cloud deploy
- A read is not a write
- Your laptop is not a public URL
- Work login is not a local key

The README is part of the API.

- 01 What does this server do, and who is it for?
- 02 Which tools are available?
- 03 How do I install and configure it?
- 04 What does a real call look like?
- 05 What permissions and risks should I understand?
- 06 What happens when it fails?
- 07 Where do I report a problem?

Screenshots are nice. A copyable path is better.

Publishing isn't the same as being findable.

I published on all four channels people use: marketplaces, npm, GitHub, IDE extensions.

What works

Searching the product name. Someone who already uses Zendesk types *"Zendesk MCP"* and lands on my server, first page.

What doesn't

Searching for what you need. Nobody finds a server by asking *"what can triage a ticket?"* – and an agent can't search at all.

Three things would fix it. **I can't build any of them** – none live in my code:

01

Search by capability

Let an agent ask *"who can triage a ticket?"* instead of needing the exact name.

needs: the protocol

02

Verified publisher

Give *"is this really from Zendesk?"* an answer you can check.

needs: the registries

03

Install at runtime

Add a server without a human hand-editing `mcp.json` first.

needs: the clients

Does it work out of the box? For a developer, and for a model.

The developer reads the README

- Running in ten minutes – timed, clean machine
- At least one **real call with real output**, not just a parameter table
- Edge cases written down: empty result, rate limit, expired token, partial failure
- Honest about limits, cost, and what it can't do
- Where to report a problem

The model reads the schema

- Names that say what they do without the source
- Descriptions that say **when** to use it, not just what it does
- Every parameter with a type, a default, and an example value
- **What "no results" means** – an empty array is not "try again". Say stop.
- Errors phrased as next actions, because it will act on them

**The model never opens your README.
The description field is the entire manual it gets.**

MCP servers can become more than integrations.

They can be the layer your team stops worrying about — because it behaves the same way every time, explains itself when it doesn't, and can be found without asking me.

Good tools.
Clear docs.
Honest metadata.
A place to find them.

Honest metadata — everything a person or an agent reads *before* installing: what it touches, which permissions it wants, what it was tested against, whether anyone still maintains it. A `1.0.0` on an abandoned server is the same lie as a `200 OK` on a broken outcome.

THANK YOU

Build one. Publish it. Make the next one easier to use.

Start with **MCP Demo**. Clone it, break it, read the refusals — every example in this talk is in there.

[MCP Demo](#) — every example from this talk · [wxo-labs](#) — 14 hands-on labs, two on MCP
No bug too small, no syntax too weird.

QUESTIONS?

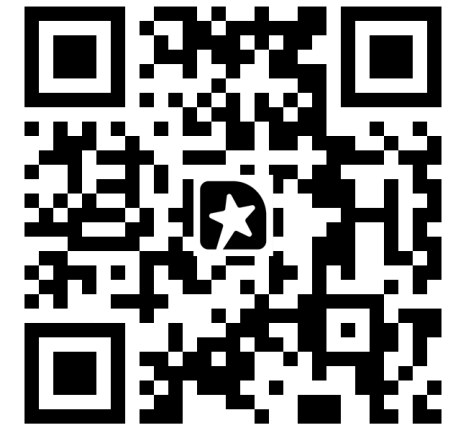
mvankempen@ca.ibm.com

markus.van.kempen@gmail.com

markusvankempen.github.io

Markus · Research · Floor 7½ 🏢👉

Backup slides follow — press → for acronyms, transports, packaging, and how I publish the next server.



MCP as a Platform: What I Learned Building a Portfolio of MCP Servers

Scan for session feedback

The acronyms.

What they actually mean.

Term	Stands for	Why it shows up in this talk
MCP	Model Context Protocol	The open standard. Clients discover and call tools; servers wrap APIs.
WYO	watsonx Orchestrate	IBM's agent orchestration platform. Labs 7–8; two servers in the portfolio.
CRN	Cloud Resource Name	IBM Cloud's globally unique ID (<code>crn:v1:...</code>). New project → new CRN → often a new hostname.
API	Application Programming Interface	The enterprise system behind the MCP server. A <code>200</code> is not a finished tool.
ADK	Agent Developer Kit	How WYO imports an MCP server as a toolkit.
CE	Code Engine	IBM Cloud serverless containers. The "cloud containers" domain.
IAM	Identity and Access Management	Why a missing tenant header fails every job while auth still looks "fine."

How MCP actually talks.

Term	What it is	Where it runs
JSON-RPC 2.0	The message format MCP speaks. Identical on every transport.	Both
stdio	The client launches your server as a child process and talks over stdin / stdout.	Local only — no port, no URL . Secrets come from env vars.
SSE Server-Sent Events	<code>text/event-stream</code> over plain HTTP. Long-lived <code>GET</code> carries server→client; a separate <code>POST</code> carries client→server.	Remote — whatever port your HTTP server binds; 443 over HTTPS in production.
Streamable HTTP	The current remote transport. One endpoint, and a response can upgrade into an SSE stream. Supersedes the older HTTP+SSE pair.	Remote — same HTTPS port.
<code>tools/list</code>	The call a client makes to enumerate what a server offers.	Both — "0 tools discovered" means this came back empty.
stderr	Where diagnostics go on stdio. <code>stdout</code> is the protocol.	Local — <code>console.log</code> corrupts JSON-RPC. Use <code>console.error</code> .

Packaging, config and distribution.

Term	What it is	Why it shows up in this talk
<code>mcp.json</code>	Client config naming a server, its command or URL, and its env vars.	Lives in <code>.vscode/</code> or <code>.cursor/</code> . The extension writes it for you.
<code>npx</code>	Runs an npm package without installing it first.	The shortest path from "found it" to "called it".
package root	Zips your server directory and uploads it so a cloud runner executes from a relative path.	The fix for 0 tools discovered on slide 9.
<code>/health</code> <code>/test</code> <code>/admin</code>	Is it alive, does it really work, can it be operated — <i>my convention, not the spec.</i>	What a deployed server needs to explain itself.
scope	A label on a credential — <code>read</code> , <code>write</code> , <code>pii</code> — naming what that credential may do.	How a server refuses with a reason instead of a bare <code>403</code> , on slide 11.
MCP Registry	The community index of published servers.	One of the four front doors on slide 14.
Open VSX	Vendor-neutral extension marketplace for VS Code-compatible editors.	Where extensions land outside the Microsoft marketplace.

Ship the package. Then ship the name.

The registry does not host your code. npm does. Publish npm first, or the name publish fails.

- 01 Bump the version in two files — `package.json` and `server.json` , same number.
- 02 Publish the package — `npm publish --access public` from `server/` , not the repo root.
- 03 Wait about a minute. npm is not instant.
- 04 Publish the name — `mcp-publisher publish` . Then open both pages.

If it has an extension: package the `.vsix` and upload it to Open VSX. The registry name must start with `io.github.youruser/` .

Questions after the talk?

Both inboxes reach me.

Markus · Research · Floor 7½ 🏢👉

Work

mvankempen@ca.ibm.com

IBM — Executive Architect & Venture Capitalist
in Residence

Old friend

markus.van.kempen@gmail.com

Open source, and anything in this deck

Portfolio

markusvankempen.github.io

Every server and extension this talk refers to

Start with [MCP Demo](#) — every example from this talk is in it. *No bug too small, no syntax too weird.*